Code Analysis – Challenge Solution: Wonderland 5

By: Tehila Chananayev

Introduction

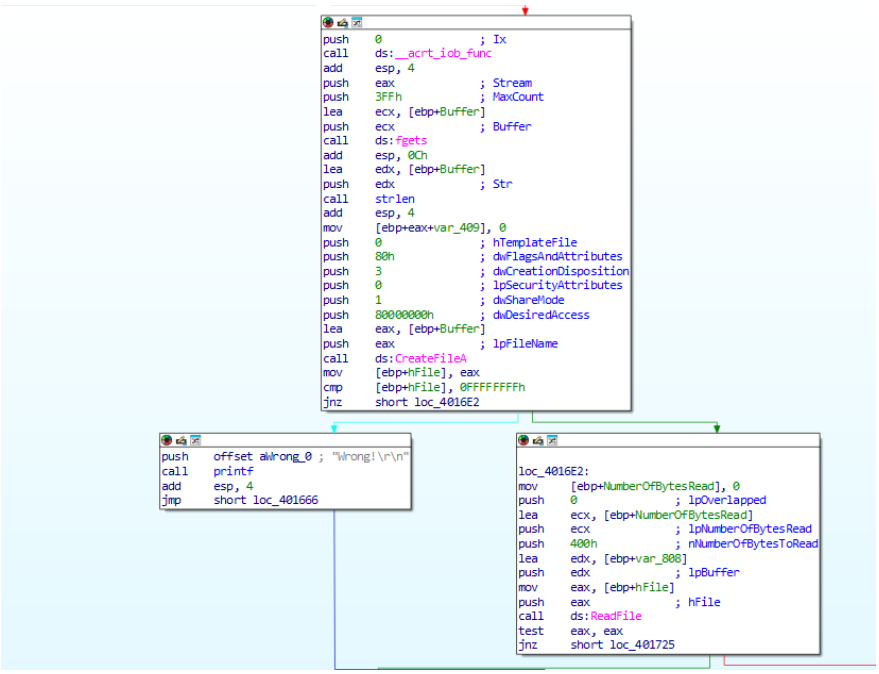
This challenge presents an assembly code snippet where the user is prompted to input the name of a file they have created. The program then loads the file's contents into memory and checks whether it contains the correct sequence.

Our goal is to perform reverse engineering on the program's logic — to understand the verification process it applies and determine the exact input (e.g., a number or string) that passes the check successfully.

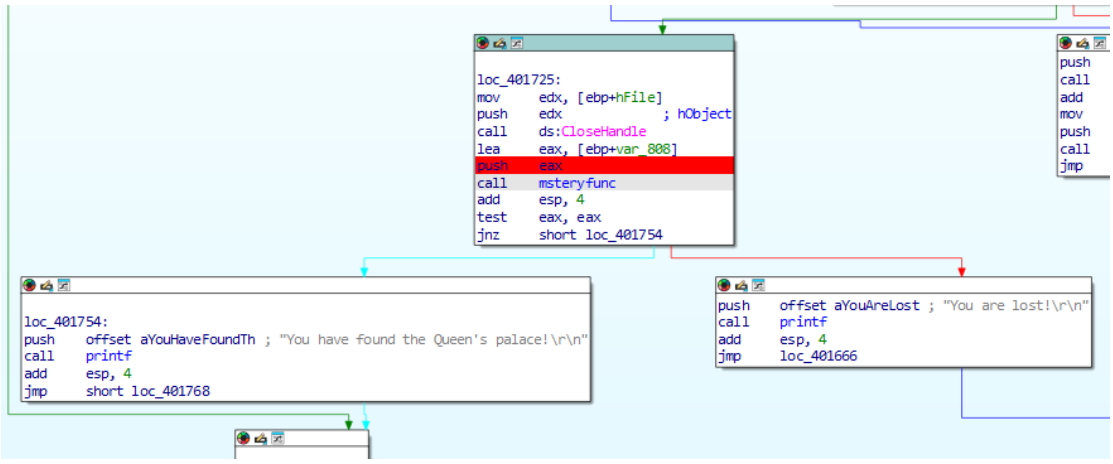
It's important to note that the solution must be achieved without any cracking or code modification, and without altering registers or memory. The challenge is solved purely through logical analysis and understanding of the assembly code.

Code Structure

Before executing the main challenge logic, the program prompts the user to input a file name via standard input. It reads the input using fgets, removes the newline character, and opens the specified file in read-only mode using CreateFileA. Then, it reads up to 1024 bytes from the file into a local buffer using the ReadFile API. If the file fails to open or read, the program does not proceed further.



Then, the program reaches the critical and most interesting part — a function I named msteryfunc. This function acts as the core challenge: if it completes successfully and returns EAX = 1 (i.e., TRUE), we win. Otherwise, if it returns EAX = 0 (FALSE), we lose.



Function msteryfunc

To analyze and understand the behavior of the function msteryfunc and determine the required input that causes it to return TRUE.

The msteryfunc function receives the contents of the file passed to eax and implements path navigation logic via an encoded string (Str) in memory. This string consists of:

'#': Walls (not allowed to be stepped on)

'.': Correct path (allowed)

'O': Starting point

'X': Destination point

Initially, the function uses the strchr function

to find the address of the character 'O' in the string and stores a pointer to it (i).

It then loops over the input string provided in user_input, and based on each character (instruction), it moves the result pointer accordingly.

```
; Attributes: bp-based frame

; BOOL __cdecl msteryfunc(int user_input)
msteryfunc proc near

var_10= dword ptr -10h
var_C= dword ptr -0Ch
var_8= dword ptr -8
i= dword ptr -4
user_input= dword ptr  8

push    ebp
mov     ebp, esp
sub     esp, 10h
push    4Fh ; 'O'        ; Val
mov     eax, Str
push    eax              ; Str
call    strchr           ; מחפשת תו במחרוזת ומחזירה את המקום הראשון של ההופעה
add     esp, 8
mov     [ebp+i], eax
mov     [ebp+var_C], 0
```

Game Rules

What happens is that there is a switch according to each instruction that appears in the contents of the file passed to the function, it does the following thing

Each input character ('1', '2', '3', '4') represents a directional command:

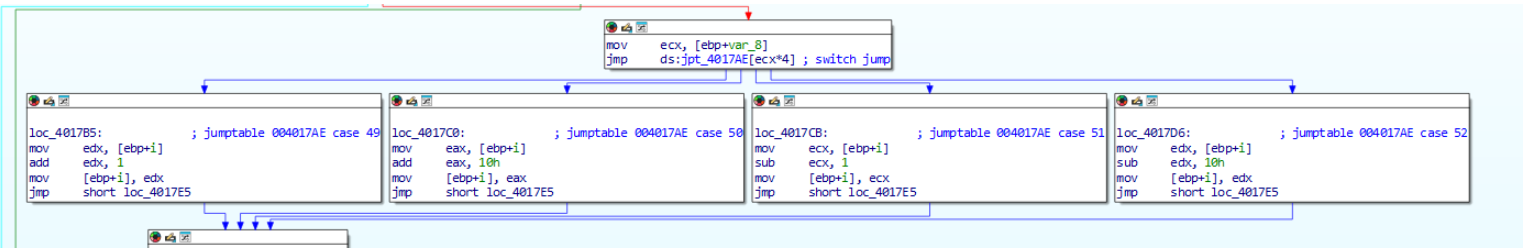
Input movement character Effect on result pointer

'1' Move result right i++

'2' Move result right (forward) i+16

'3' Move result left i--

'4' Move result left (backward) i-16



What Does the Function Do?

It simulates a **navigation puzzle** through a memory-based map (Str), starting at 'O', and following a sequence of moves given as characters in the input.

Each move must land on a '.' (dot). If the moves eventually reach 'X' exactly when the input ends — the function returns TRUE.

If the path hits a wall ('#') or finishes in the wrong place — it returns FALSE.

```
.text:004017E5
.text:004017E5 loc_4017E5:
.text:004017E5 mov     eax, [ebp+var_C]
.text:004017E8 add     eax, 1
.text:004017EB mov     [ebp+var_C], eax
.text:004017EE mov     ecx, [ebp+user_input]
.text:004017F1 add     ecx, [ebp+var_C]
.text:004017F4 movsx   edx, byte ptr [ecx]
.text:004017F7 test    edx, edx
.text:004017F9 jz      short loc_401806

.text:004017FB mov     eax, [ebp+i]
.text:004017FE movsx   ecx, byte ptr [eax]
.text:00401801 cmp     ecx, 2Eh ; '.'
.text:00401804 jz      short loc_401790

.text:00401806
.text:00401806 loc_401806:
.text:00401806 mov     edx, [ebp+i]
.text:00401809 movsx   eax, byte ptr [edx]
.text:0040180C cmp     eax, 58h ; 'X'
.text:0040180F jnz     short loc_40181A
```

This is essentially the string str where everything is done in a maze and the goal is to reach x according to the rules.

```
00404020  23 23 23 23 23 23 23 23  23 23 23 23 23 23 23 23  ################
00404030  23 23 23 4F 23 23 23 23  2E 2E 2E 23 23 23 23 23  ###O####...#####
00404040  23 23 23 2E 23 23 2E 2E  2E 23 2E 23 23 23 23 23  ###.##...#.#####
00404050  23 23 23 2E 23 23 2E 23  23 23 2E 23 23 23 23 23  ###.##.###.#####
00404060  23 23 23 2E 2E 2E 2E 23  23 23 2E 2E 2E 58 23 23  ###....###...X##
00404070  23 23 23 23 23 23 23 23  23 23 23 23 23 23 23 23  ################
00404080  00 00 00 00 45 6D 70 74  79 00 00 00 00 00 00 00  ....Empty.......
```

In the file I put the series of commands that leads to success

go.txt

```
2221114411411222111
```

```
Welcome to Wonderland. I am the mad hatter, and I have some riddles for you...
Input a level number (latest level- 5):
5


You may enter, but can you find the Queen's palace?...
(You're not a noob by now. Figure it out on your own.)
Wrong!
go.txt
You have found the Queen's palace!
```